Neural Networks

Mamta Bhattarai

MBUST

July 16, 2026

Artificial Neural Networks (ANN)

Artificial Neural Networks (ANNs) are computational models inspired by the way neurons in the human brain process information. They learn by adjusting internal parameters (weights and biases) so that the predicted output becomes closer to the actual output.

Key Characteristics

- Consist of interconnected processing units called **neurons**.
- Learn directly from training data instead of using manually designed rules.
- Capable of modeling both linear and highly nonlinear relationships.
- Performance generally improves with more training data.

Applications

- Image and face recognition
- Speech recognition and voice assistants
- Natural language processing and translation
- Medical diagnosis and disease prediction
- Financial forecasting and fraud detection

Biological Neuron vs Artificial Neuron

Artificial neurons are simplified mathematical models of biological neurons.

Biological Neuron	**Artificial Neuron**
Dendrites receive signals	Input features $(x_1, x_2, \ldots, x_n)$
Cell body processes signals	Weighted summation $(\sum w_i x_i + b)$
Synapses determine signal strength	Weights (w_i)
Threshold determines firing	Activation function
Axon transmits output	Output prediction (y)

Artificial neurons imitate the basic idea of receiving information, processing it, and producing an output.

Perceptron Model

The perceptron is the simplest form of an artificial neural network and is designed for binary classification problems.

$z = \sum_{i=1}^{n} w_i x_i + b$
$y = f(z)$
where

- x_i : Input features
- w_i : Learnable weights indicating feature importance
- b : Bias that shifts the decision boundary
- z : Weighted sum of inputs
- $f(z)$: Activation function producing the final output

A perceptron learns the optimal values of weights and bias during training.

Working of a Perceptron

The perceptron transforms input data into an output prediction through a sequence of simple computations.

1. Receive input features.
2. Multiply each input by its corresponding weight.
3. Add all weighted inputs and the bias.
4. Apply an activation function.
5. Produce the predicted output.

$\boxed{\text{Inputs}} \rightarrow \boxed{\text{Weighted Sum}} \rightarrow \boxed{\text{Activation}} \rightarrow \boxed{\text{Prediction}}$

Each training iteration updates the weights to reduce prediction errors.

Why Do We Need Activation Functions?

Consider a neural network without activation functions.
$y = W_2(W_1x + b_1) + b_2$
This can be rewritten as
$y = Wx + b$
where $W = W_2W_1$.

Observation
No matter how many layers are added, the network behaves like a single linear model.

Why is this a problem?

- Cannot learn complex patterns.
- Cannot solve non-linear problems.
- Performs poorly on image, speech and language tasks.

Activation functions introduce non-linearity, enabling neural networks to model complex relationships in data.

What is an Activation Function?

An activation function determines whether a neuron should be activated.

Each neuron first computes
$z = \sum_{i=1}^{n} w_i x_i + b$
The activation function transforms this value into
$a = f(z)$
where

- z : weighted sum
- $f(z)$: activation function
- a : neuron output

Inputs $\rightarrow$ Weighted Sum $\rightarrow$ Activation Function $\rightarrow$ Output

Properties of a Good Activation Function

An ideal activation function should possess several important characteristics.

- Introduce non-linearity.
- Be computationally efficient.
- Have a smooth and differentiable curve.
- Avoid vanishing or exploding gradients.
- Allow fast convergence during training.
- Produce stable outputs.
- Support effective backpropagation.

Different activation functions satisfy these properties to different degrees, making each suitable for specific applications.

Common Activation Functions

Activation	**Range**	**Common Use**
Binary Step	0 or 1	Perceptron
Sigmoid	(0,1)	Binary classification
Tanh	(-1,1)	Hidden layers (older networks)
ReLU	$(0,\infty)$	Deep learning
Leaky ReLU	$(-\infty,\infty)$	Deep learning
ELU	$(-1,\infty)$	Deep networks
Softmax	Probability	Multi-class classification

Binary Step Activation Function

The binary step function is the simplest activation function.

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Characteristics

- Produces only two outputs.
- Suitable for simple binary decisions.
- Used in the original perceptron.
- Not differentiable.
- Cannot be used with gradient descent.

Application

Simple threshold-based decision systems.

Sigmoid Activation Function

The sigmoid function maps any real value into the interval (0,1).
$f(x) = \frac{1}{1+e^{-x}}$

Characteristics

- Smooth and differentiable.
- Output interpreted as probability.
- Commonly used for binary classification.
- Large positive values approach 1.
- Large negative values approach 0.

Advantages

- Smooth learning.
- Probabilistic interpretation.

Disadvantages

- Vanishing gradient.
- Slow convergence.
- Output not zero-centered.

Hyperbolic Tangent (Tanh)

The tanh function is a scaled version of sigmoid.
$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Output range
$(-1, 1)$

Advantages

- Zero-centered output.
- Faster convergence than sigmoid.
- Smooth gradient.

Limitations

- Suffers from vanishing gradient.
- Less common in modern deep networks.

ReLU (Rectified Linear Unit)

ReLU is the most widely used activation function in deep learning.
$f(x) = \max(0, x)$

Characteristics

- Positive inputs remain unchanged.
- Negative inputs become zero.
- Computationally inexpensive.
- Faster training.
- Sparse neuron activation.

Advantages

- Reduces vanishing gradients.
- Excellent performance for deep networks.
- Simple implementation.

Limitation

Dead ReLU problem.

Leaky ReLU

Leaky ReLU addresses the dead neuron problem of ReLU.

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

where

$\alpha = 0.01$

Advantages

- Allows small negative gradients.
- Prevents neurons from dying.
- Better gradient flow.

Disadvantage

Requires choosing parameter α.

ELU (Exponential Linear Unit)

ELU improves learning by allowing negative outputs.

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

Advantages

- Faster convergence.
- Mean activations closer to zero.
- Reduces bias shift.
- Better than ReLU in some deep models.

Disadvantage

More computationally expensive than ReLU.

Softmax Activation Function

Softmax converts outputs into probabilities.
Suppose
$z = (z_1, z_2, \ldots, z_n)$
then
$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$

Properties

- Outputs are probabilities.
- Values lie between 0 and 1.
- Sum of probabilities equals 1.
- Used in multi-class classification.

Comparison of Activation Functions

Function	Range	Zero Centered	Vanishing Gradient	Common Use
Binary Step	0,1	No	Yes	Perceptron
Sigmoid	(0,1)	No	Yes	Binary Output
Tanh	(-1,1)	Yes	Yes	Hidden Layers
ReLU	$(0,\infty)$	No	No	Deep Learning
Leaky ReLU	All	No	Less	Deep Learning
ELU	$(-1,\infty)$	Nearly	Less	Deep Learning
Softmax	(0,1)	No	No	Multi-class Output

Which Activation Function Should We Choose?

Problem	**Recommended Function**
Hidden layers	ReLU
Very deep networks	Leaky ReLU / ELU
Binary classification output	Sigmoid
Multi-class classification output	Softmax
Older shallow networks	Tanh
Simple threshold decisions	Binary Step

Practical Recommendation

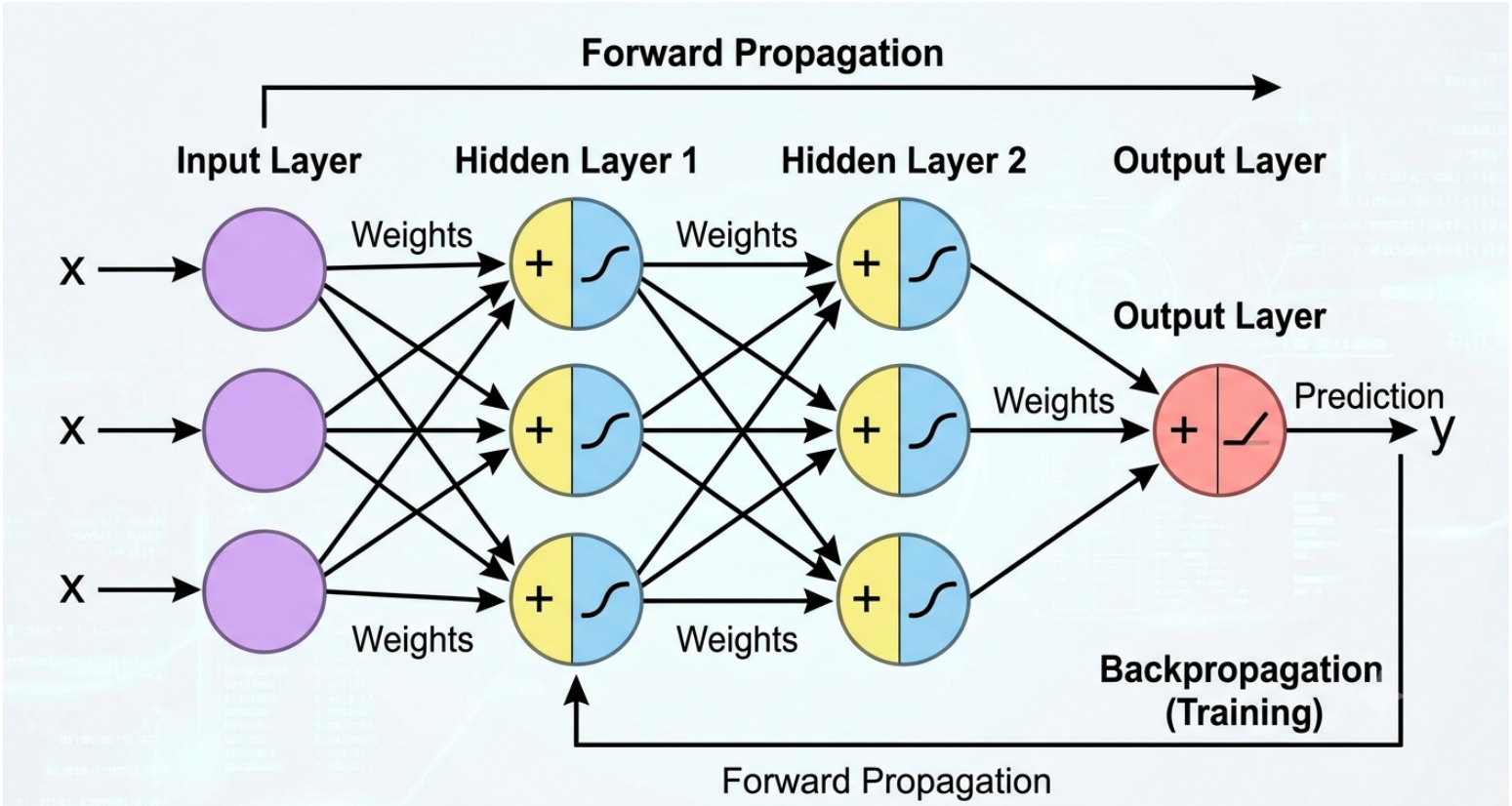
- Use ReLU for most hidden layers.
- Use Sigmoid for binary output neurons.
- Use Softmax for multi-class output neurons.
- Consider Leaky ReLU or ELU when dead ReLU becomes a problem.

Summary

- Activation functions introduce non-linearity.
- They determine whether neurons become active.
- Different activation functions have different strengths.
- ReLU is the most commonly used hidden-layer activation.
- Sigmoid is preferred for binary classification.
- Softmax is used for multi-class classification.
- Choosing the correct activation function greatly influences learning speed and model performance.

Multilayer Perceptron (MLP)

A **Multilayer Perceptron (MLP)** is a type of **feedforward artificial neural network** that consists of an input layer, one or more hidden layers, and an output layer. By introducing hidden layers with nonlinear activation functions, an MLP can learn complex relationships between inputs and outputs.



Architecture of a Multilayer Perceptron (MLP)

An MLP consists of multiple fully connected layers, where each neuron in one layer is connected to every neuron in the next layer. Hidden layers use activation functions to learn nonlinear relationships within the data.

Architecture

- **Input Layer:** Receives the input features.
- **Hidden Layer(s):** Learn increasingly abstract feature representations through weighted connections and activation functions.
- **Output Layer:** Produces the final prediction (classification or regression).

Key Advantage

Unlike a single-layer perceptron, an MLP can learn nonlinear decision boundaries, enabling it to solve complex classification and regression problems.

Training Neural Networks

Training aims to minimize prediction errors by continuously updating the model parameters.

Training Process

1. Perform forward propagation.
2. Compute the loss function.
3. Use backpropagation to calculate gradients.
4. Update weights using an optimization algorithm.
5. Repeat until the model converges.

Common Optimizers

- Gradient Descent
- Stochastic Gradient Descent (SGD)
- Adam Optimizer

Gradient Descent (GD)

Gradient Descent is an optimization algorithm used to minimize the loss function by updating model parameters in the direction of the negative gradient.

$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$

where

- w: Model parameter (weight)
- η: Learning rate
- L: Loss function

Characteristics

- Uses the **entire training dataset** to compute the gradient.
- Produces stable and smooth updates.
- Computationally expensive for large datasets.
- One weight update per epoch.

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent updates the model parameters using **one training example at a time**.

$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L_i}{\partial w}$

where L_i is the loss for a single training sample.

Characteristics

- Updates weights after every training example.
- Faster updates than batch gradient descent.
- Requires less memory.
- Gradient is noisy, causing fluctuations during learning.
- Often escapes local minima more effectively.

Adam Optimizer: Formulation

Adam (Adaptive Moment Estimation) combines the advantages of **Momentum** (first-order moment) and **RMSProp** (adaptive learning rate).
Given the gradient at iteration t,
$g_t = \nabla_w L(w_t)$,
Adam computes two exponentially weighted moving averages:

First Moment (Momentum)

$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$

where

- m_t : moving average of gradients
- β_1 : decay rate for first moment (typically 0.9)

Second Moment (RMSProp)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where

- v_t : moving average of squared gradients
- β_2 : decay rate for second moment (typically 0.999)

Adam Optimizer: Bias Correction and Weight Update

Since m_t and v_t are initialized to zero, they are biased during the initial iterations.

Bias-Corrected Estimates

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

where

- $\hat{m}_t$: bias-corrected first moment estimate
- $\hat{v}_t$: bias-corrected second moment estimate
- t : iteration number

Weight Update

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Comparison of Optimization Algorithms

Feature	**GD**	**SGD**	**Adam**
Data used per update	Entire dataset	One sample	Mini-batch
Update frequency	Once/epoch	Every sample	Every mini-batch
Training speed	Slow	Fast	Very Fast
Memory usage	High	Low	Moderate
Gradient stability	Stable	Noisy	Stable
Adaptive learning rate	No	No	Yes
Suitable for deep learning	Limited	Good	Excellent

Which Optimizer Should You Use?

- **Gradient Descent**
  - Best for small datasets.
  - Stable but computationally expensive.
- **Stochastic Gradient Descent (SGD)**
  - Faster learning.
  - Often preferred when generalization is important.
- **Adam**
  - Default choice for most deep learning applications.
  - Faster convergence with adaptive learning rates.
  - Works well on large and complex datasets.

GD $\rightarrow$ Small Data

SGD $\rightarrow$ Fast Training

Adam $\rightarrow$ Deep Learning

Multilayer Perceptron (MLP): Learning Process

An MLP learns by repeating two main steps:

1. **Forward Pass**
   - Input data is passed through the network.
   - Activations are computed layer by layer.
   - Final output prediction is generated.
2. **Backward Pass (Backpropagation)**
   - Error between predicted and actual output is calculated.
   - Gradients of weights are computed using chain rule.
   - Weights are updated to reduce the error.

Input $\rightarrow$ Forward Pass $\rightarrow \hat{y} \rightarrow$ Loss $\rightarrow$ Backpropagation $\rightarrow$ Updated Weights

Architecture of a Simple MLP

Consider an MLP with:

- One input neuron
- One hidden neuron
- One output neuron
- Sigmoid activation function

$$x \to \boxed{w_1, b_1} \to h \to \boxed{w_2, b_2} \to \hat{y}$$

For each neuron:
$z = wx + b$
$a = \sigma(z) = \frac{1}{1+e^{-z}}$
where:
$z =$ weighted input, $\quad a =$ activation

Forward Pass in MLP

Forward pass computes the prediction.

Step 1: Hidden layer

$z_h = w_1 x + b_1$

Apply activation:

$a_h = \sigma(z_h)$

Step 2: Output layer

$z_o = w_2 a_h + b_2$

Final prediction:

$\hat{y} = \sigma(z_o)$

Goal of Forward Pass

Convert input features into a prediction using current weights.

Loss Function

The network compares prediction with the true value.
For regression:
$E = \frac{1}{2}(y - \hat{y})^2$
where:
$y =$ actual output
$\hat{y} =$ predicted output
Example:
$y = 1, \qquad \hat{y} = 0.56$
$E = \frac{1}{2}(1 - 0.56)^2$
A large error means weights need more correction.

Backpropagation: Main Idea

Backpropagation sends the error backward through the network.

$$\boxed{Error} \leftarrow \boxed{OutputLayer} \leftarrow \boxed{HiddenLayer} \leftarrow \boxed{Input}$$

It calculates:
$\frac{\partial E}{\partial w}$
which tells:

- How much each weight contributed to the error.
- Direction in which the weight should move.

Weights are updated using:
$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$

Sigmoid Derivative

Sigmoid function:
$\sigma(z) = \frac{1}{1+e^{-z}}$
Derivative:
$$\boxed{\sigma'(z) = \sigma(z)(1 - \sigma(z))}$$
During backpropagation:
Gradient = Error $\times$ Sigmoid derivative
Example:
If:
$\sigma(z) = 0.6$
then:
$\sigma'(z) = 0.6(1 - 0.6)$
$= 0.24$

Numerical Example: Given Parameters

Network:

$x = 1, \qquad y = 1$

Weights:

$w_1 = 0.5, \qquad w_2 = 0.4$

Bias:

$b_1 = b_2 = 0$

Learning rate:

$\eta = 0.1$

Loss:

$E = \frac{1}{2}(y - \hat{y})^2$

Numerical Example: Forward Pass

Hidden neuron:

$z_h = w_1 x + b_1$

$= 0.5(1) = 0.5$

Activation:

$a_h = \sigma(0.5) = 0.6225$

Output neuron:

$z_o = w_2 a_h + b_2$

$= 0.4(0.6225) = 0.249$

Prediction:

$\hat{y} = \sigma(0.249)$

$\boxed{\hat{y} = 0.5619}$

Numerical Example: Compute Error

$E = \frac{1}{2}(y - \hat{y})^2$

Substitute:

$E = \frac{1}{2}(1 - 0.5619)^2$

$\boxed{E = 0.0959}$

The goal of backpropagation is to reduce this error.

Numerical Example: Output Layer Gradient

Output error:

$\delta_o = (\hat{y} - y)\sigma'(z_o)$

$= (0.5619 - 1)(0.5619)(1 - 0.5619)$

$\boxed{\delta_o = -0.1079}$

Gradient:

$\frac{\partial E}{\partial w_2} = \delta_o a_h$

$= (-0.1079)(0.6225)$

$$\boxed{\frac{\partial E}{\partial w_2} = -0.0672}$$

Numerical Example: Hidden Layer Gradient

Hidden error:

$\delta_h = \delta_o w_2 \sigma'(z_h)$

$= (-0.1079)(0.4)(0.6225)(1 - 0.6225)$

$\boxed{\delta_h = -0.0101}$

Gradient:

$\frac{\partial E}{\partial w_1} = \delta_h x$

$$\boxed{\frac{\partial E}{\partial w_1} = -0.0101}$$

Weight Update

Gradient descent:

$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$

Output weight:

$w_2 = 0.4 - 0.1(-0.0672)$

$\boxed{w_2 = 0.4067}$

Hidden weight:

$w_1 = 0.5 - 0.1(-0.0101)$

$\boxed{w_1 = 0.5010}$

After one iteration

The error decreases and the network moves closer to the correct prediction.